



UNIVERSIDAD POLITÉCNICA INTERNACIONAL

Técnicas de Programación

La Pachanga del Mundial

Documentación Técnica Final

Proyecto #2

Estudiante: Erick Delgado Araya

Profesor: Luis Felipe Mora Umaña

Índice

Índice.....	1
Introducción.....	2
Decisiones de Diseño	3
Migración de Windows Forms a Blazor Server	3
Patrón Singleton para la gestión de sesión.....	3
Control de roles (Usuario vs Administrador)	4
Sistema de testing con xUnit	4
Descarga de reportes desde el navegador	5
Desarrollo.....	6
Requisitos para ejecutar el sistema	6
Estructura del proyecto	6
Cómo ejecutar el sistema.....	7
Interacciones del sistema	7
Login.....	7
Registro de usuario	7
Dashboard de usuario	7
Panel de Administración	8
Gestionar Partidos (Admin).....	8
Gestionar Usuarios (Admin).....	8
Partidos	8
Quinielas	8
Pronósticos.....	9
Estadísticas.....	9
Bibliografía	10
Análisis de Aprendizaje	11

Introducción

Este documento es la recopilación final de la documentación técnica del Proyecto #2 en la asignatura de Técnicas de Programación. Este trabajo constituye la segunda fase de "La Pachanga del Mundial", un sistema que gestiona apuestas para el Mundial de la FIFA 2026. En esta actualización, se trasladaron las características implementadas en el Proyecto #1 a una nueva arquitectura web, añadiendo nuevas funciones y un sistema de pruebas unitarias.

Las modificaciones más relevantes en comparación con el Proyecto #1 incluyen: el traslado de la interfaz de Windows Forms a Blazor Server, la adición de un usuario administrador con privilegios especiales, notificaciones al iniciar sesión sobre los partidos próximos a predecir, informes descargables en formatos CSV y TXT, así como un total de 40 pruebas unitarias utilizando xUnit que abordan los cinco controladores más importantes del sistema.

Los Modelos y Controladores del Proyecto #1 se conservaron sin alteraciones en su estructura, manteniendo el principio de separación de capas del patrón MVC; únicamente se sustituyó la capa de Vista (de Windows Forms a Blazor). Esta elección reafirma la calidad del diseño inicial y resalta la capacidad de expansión del sistema.

Decisiones de Diseño

Migración de Windows Forms a Blazor Server

La decisión principal en esta fase fue sustituir la capa de presentación (Vistas) de Windows Forms por Blazor Server. Blazor posibilita la creación de interfaces web dinámicas utilizando C# en vez de JavaScript, lo que hace más sencilla la adaptación para un programador familiarizado con .NET sin la necesidad de aprender un nuevo lenguaje.

Cada formulario de Windows Forms (.cs + .Designer.cs) fue transformado en un componente Razor (.razor), que integra HTML para el diseño visual y C# para la lógica dentro de un bloque `@code { }`. La correspondencia entre tecnologías es clara: `@bind` sustituye a `TextBox`, `@onclick` reemplaza al evento `Click`, y `NavigationManager.NavigateTo` reemplaza la apertura de nuevos formularios.

Se optó por Blazor Server (en vez de Blazor WebAssembly) porque permite que el código C# opere en el servidor, lo que simplifica el manejo del estado de la sesión y el acceso a los archivos JSON de datos sin requerir APIs adicionales.

Patrón Singleton para la gestión de sesión

En Windows Forms el usuario que está utilizando la aplicación se almacenaba como una variable en el `MainForm`. En Blazor, dado que cada componente opera de manera autónoma, es necesario hallar un método para compartir al usuario actual entre todas las páginas. Para lograr esto, se creó la clase `SesionUsuario`, que se registró como Singleton en `Program.cs`.

Un Singleton es un patrón de diseño que asegura que únicamente haya una instancia de la clase durante todo el ciclo de vida de la aplicación. Al registrar `SesionUsuario` como un

Singleton, cualquier componente Razor puede inyectarla a través de @inject y acceder al usuario activo sin la necesidad de transferirlo como argumento entre páginas. Esto unifica la lógica de autenticación y facilita la gestión de acceso en cada página.

Control de roles (Usuario vs Administrador)

Se incorporó un atributo EsAdministrador al modelo Usuario, así como un usuario especial denominado 'admin' en el documento usuarios.json. Al momento de iniciar sesión, el sistema evalúa este atributo y dirige al usuario a la interfaz adecuada: AdminDashboard para los administradores y Home para los usuarios regulares.

Cada página protegida verifica al inicio si el usuario actual posee el rol adecuado mediante la expresión !Sesion.EstaLogueado || !Sesion.EsAdministrador, redirigiendo al inicio de sesión en caso de que la entrada no sea autorizada. El administrador tiene acceso exclusivo a la gestión de los partidos (modificación de resultados y estados) y la administración de usuarios (restablecimiento de contraseñas, activación y desactivación). El administrador no está habilitado para hacer pronósticos.

Sistema de testing con xUnit

Se estableció un proyecto independiente llamado LaPachangaDelMundialV2.Tests utilizando el framework xUnit, que es el más indicado para realizar pruebas unitarias en .NET. Este proyecto se vincula al proyecto principal para poder acceder a los modelos y controladores.

Se llevaron a cabo 40 pruebas unitarias organizadas en 5 clases de pruebas, una para cada uno de los controladores principales del sistema. Cada prueba sigue la estructura de preparar, ejecutar y verificar: en la preparación se establecen los datos de entrada, en la ejecución se

llama al método que se está evaluando y en la verificación se confirma que el resultado obtenido sea el correcto. Se utilizó `DateTime.Now.Ticks` para generar identificadores únicos en las pruebas que alteran datos, minimizando conflictos en ejecuciones sucesivas.

La cobertura estimada alcanza más del 60% de la capa de lógica de negocio, abarcando los métodos esenciales de cada controlador: manejo de autenticación y registro de usuarios, cálculo de la tabla de posiciones, registro y puntuación de predicciones, administración de quinielas y cálculo de estadísticas.

Descarga de reportes desde el navegador

Para llevar a cabo la descarga de archivos en Blazor Server, se utilizó la Interoperabilidad de JavaScript (IJSRuntime), lo que facilita la invocación de funciones de JavaScript desde el código C#. En el archivo `wwwroot/descargar.js` se diseñó una función llamada `descargarCSV` que crea un enlace de descarga de manera dinámica, conteniendo el contenido del informe.

Se ofrecen dos tipos de archivos para descargar: CSV, que es adecuado para su uso en Excel y programas de análisis de datos, y TXT, que tiene un formato claro y fácil de leer. El nombre del archivo refleja el rango de fechas elegido, lo que ayuda en su identificación. Los informes se generan en memoria utilizando los datos procesados por el `EstadisticasController`, lo que elimina la necesidad de archivos temporales en el servidor.

Desarrollo

Requisitos para ejecutar el sistema

- Sistema operativo: Windows 10 o superior.
- Visual Studio 2022 con workload de desarrollo web ASP.NET.
- .NET 8.0 SDK instalado.
- Paquete NuGet: Newtonsoft.Json (se instala automáticamente al abrir el proyecto).
- Conexión a internet para cargar Bootstrap desde CDN.

Estructura del proyecto

El repositorio mantiene la rama master con el Proyecto #1 (Windows Forms) y la rama v2 con el Proyecto #2 (Blazor). La estructura del proyecto v2 es:

Carpeta / Archivo	Descripción
docs/	Documentación (Entregable #1, doc. final v1 y v2)
src/LaPachangaDelMundialV2/	Proyecto principal Blazor Server
src/.../Models/	Clases del dominio (sin cambios respecto al Proyecto #1)
src/.../Controllers/	Lógica del sistema (sin cambios respecto al Proyecto #1)
src/.../Utils/	Páginas Blazor: Login, Home, Partidos, Quinielas, Pronosticos, Estadísticas, AdminDashboard, AdminPartidos, AdminUsuarios
src/.../Components/Pages/	Clases de apoyo: JsonLoader y SistemaFecha
src/.../Datos/	Archivos JSON de carga inicial
src/.../wwwroot/	Archivos estáticos: CSS, JS (descargar.js)
LaPachangaDelMundialV2.Tests/	Proyecto de pruebas unitarias xUnit

Cómo ejecutar el sistema

Abrir el archivo LaPachangaDelMundial.sln en Visual Studio 2022, asegurarse de estar en la rama v2 de Git, establecer LaPachangaDelMundialV2 como proyecto de inicio y presionar F5. El navegador se abrirá automáticamente en <https://localhost> con la página de Login. Para ejecutar las pruebas: ir a Prueba → Ejecutar todas las pruebas.

Interacciones del sistema

Login

Página de inicio del sistema. El usuario ingresa sus credenciales y el sistema valida contra el archivo usuarios.json. Si las credenciales son incorrectas muestra un mensaje de error en rojo. Si el usuario está desactivado muestra el mensaje 'Tu cuenta está desactivada. Contacta al administrador'. Si el login es exitoso redirige al Dashboard del usuario o al Panel de Administración según el rol.

Registro de usuario

Accesible desde el Login. Permite crear una nueva cuenta ingresando nombre de usuario (mínimo 4 caracteres), contraseña (mínimo 6 caracteres) y país preferido seleccionado de un listado con las 48 selecciones del Mundial 2026. El sistema valida que el nombre no esté en uso y guarda el nuevo usuario en el JSON.

Dashboard de usuario

Muestra el perfil del usuario (nombre, país, puntos e insignias), un menú de acceso a todos los módulos y un control de fecha simulada del sistema. Si hay partidos pendientes de pronosticar en las próximas 24 horas, muestra una alerta amarilla con el listado de esos partidos y un acceso directo a Pronósticos.

Panel de Administración

Dashboard exclusivo para el usuario administrador. Muestra accesos a Gestionar Partidos, Gestionar Usuarios y Estadísticas. No tiene acceso a Pronósticos ni Quinielas.

Gestionar Partidos (Admin)

Lista todos los partidos del sistema con su fecha, equipos, marcador, fase y estado. Al hacer click en 'Editar' sobre un partido, se despliega un formulario para modificar el marcador, el estado (Pendiente/EnCurso/Finalizado) y la fecha y hora del partido. Los cambios se aplican en memoria inmediatamente.

Gestionar Usuarios (Admin)

Lista todos los usuarios del sistema (excepto el admin). Para cada usuario permite resetear su contraseña (el admin elige la nueva contraseña) o activar/desactivar la cuenta. Un usuario desactivado no puede iniciar sesión. Los cambios se guardan en el archivo usuarios.json.

Partidos

Mismo funcionamiento que en el Proyecto #1 pero en formato web con 4 pestañas: Últimos 5 partidos finalizados, Próximos 24 horas, Tabla de Grupos (con selector de grupo A-L) y Fase Eliminatoria (Dieciseisavos, Octavos, Cuartos, Semifinales, Tercer Puesto y Final). Incluye un botón para recargar los datos con la fecha simulada actual.

Quinielas

Permite ver las quinielas del usuario con su timeline de notificaciones, crear nuevas quinielas (privadas o públicas) y unirse a quinielas existentes donde el usuario no sea integrante.

Pronósticos

Tres pestañas: Pronosticar (lista partidos pendientes con inputs de marcador, marca con los ya pronosticados), ✓ Mi Historial (tabla con pronósticos, resultado real y puntos obtenidos) y Ranking (global o por quiniela, ordenado de mayor a menor puntaje).

Estadísticas

Permite seleccionar un rango de fechas y calcular 7 indicadores: equipo más apostado, resultado más repetido, partido con más aciertos, usuario con más aciertos, partido con más pronósticos, promedio de goles y equipo sorpresa. Incluye botones para descargar el reporte en formato CSV o TXT con el rango de fechas en el nombre del archivo.

Bibliografía

FIFA. (2026). Copa Mundial de la FIFA 2026 — Resultados y estadísticas. Recuperado de <https://www.fifa.com/es>

Microsoft. (2024). Documentación de ASP.NET Core Blazor. Recuperado de <https://learn.microsoft.com/es-es/aspnet/core/blazor/>

Microsoft. (2024). Pruebas unitarias en .NET con xUnit. Recuperado de <https://learn.microsoft.com/es-es/dotnet/core/testing/unit-testing-with-dotnet-test>

Bootstrap. (2024). Bootstrap 5 Documentation. Recuperado de <https://getbootstrap.com/docs/5.3/>

SonarSource. (2024). SonarLint for Visual Studio. Recuperado de <https://www.sonarsource.com/products/sonarlint/>

Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.

Análisis de Aprendizaje

El desarrollo del Proyecto #2 validó uno de los conceptos más cruciales del diseño del Proyecto #1: que aplicar correctamente una arquitectura MVC facilita que la transición de tecnología en la capa de presentación sea un procedimiento organizado y predecible. Los Controladores y Modelos no necesitaban modificaciones estructurales para operar con Blazor; únicamente se requirió sustituir la capa de Vista, tal como se había concebido desde el principio.

El aprendizaje más relevante de esta etapa fue entender el funcionamiento del estado en una aplicación web frente a una aplicación de escritorio. En Windows Forms, el estado residía en las variables del formulario. En Blazor Server, donde cada componente actúa de manera independiente, fue necesario aplicar el patrón Singleton para compartir la sesión del usuario entre distintas páginas. Este cambio de perspectiva —de 'estado local en el formulario' a 'estado compartido en un servicio'— es clave para el desarrollo web contemporáneo.

La introducción de un sistema de pruebas unitarias con xUnit también significó un avance significativo. El patrón Arrange-Act-Assert proporciona una estructura clara para cada prueba y hace más sencillo comprender qué se evalúa y por qué. Un desafío que se presentó fue que los controladores accedían a archivos JSON en el disco, lo que ocasionó errores cuando se realizaban las pruebas en un entorno diferente al de la aplicación. Esto se solucionó implementando el manejo de excepciones en el método Guardar() y utilizando DateTime.Now.Ticks para crear identificadores únicos en cada ejecución de prueba.

En resumen, el sistema cumplió con todas las necesidades fundamentales del Proyecto #2: migración a Blazor Server, cuenta de administrador con gestión de partidos y usuarios,

notificaciones de partidos próximos al iniciar sesión, reportes descargables en formatos CSV y TXT, y 40 pruebas unitarias con una cobertura estimada de más del 60% de la lógica de negocio. El proyecto se establece como una base sólida para futuras iteraciones, donde se podría integrar una base de datos relacional, sustituyendo únicamente la capa de acceso a datos (JsonLoader) sin interferir con el resto del sistema.